



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

[Hardening de Gnu/Linux]

P R E S E N T A:

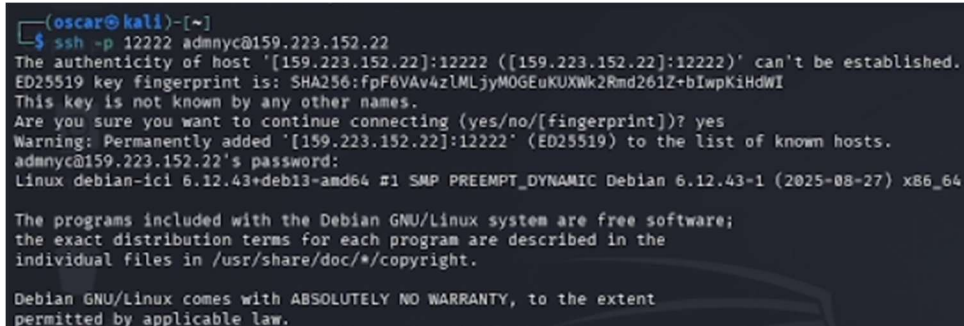
**[Oscar Alberto Leal Ramírez][22590042]
[Isaac Castro Islas][B19140346]
[Hortencia Sanchez Carlos][B23590532]
[Ingeniería en Sistemas Computacionales]**

PERIODO [Enero-Junio (2026)]

Paso 1: Conexión remota por SSH e intercambio de llaves:

Comando ejecutado:

ssh -p 12222 [admnyc@159.223.152.22](#)



```
(oscar@kali)-[~]
$ ssh -p 12222 admnyc@159.223.152.22
The authenticity of host '[159.223.152.22]:12222 ([159.223.152.22]:12222)' can't be established.
ED25519 key fingerprint is: SHA256:FpF6VAv4zLMLjyMOGEuKUXWk2Rmd261Z+bIwpKiHdWI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[159.223.152.22]:12222' (ED25519) to the list of known hosts.
admnyc@159.223.152.22's password:
Linux debian-ici 6.12.43+deb13+amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.43-1 (2025-08-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

Figura 1. Proceso de autenticación e inicio de sesión SSH.

Se realizó la conexión por SSH desde Kali hacia el servidor Debian utilizando el puerto 12222. Posteriormente, se aceptó la huella digital del servidor y se inició sesión con el usuario admnyc en lugar de root. Estas acciones se implementaron para mejorar la seguridad, ya que el cambio de puerto reduce ataques automáticos al puerto 22, la verificación del fingerprint protege contra ataques Man-in-the-Middle y el uso de un usuario estándar evita el acceso directo como administrador.

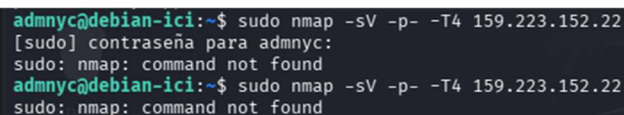
¿Por qué usamos Máquinas Virtuales (VM)?

Se utilizaron máquinas virtuales para crear un entorno de pruebas seguro y controlado. Kali Linux permitió realizar auditorías y pruebas sin afectar el sistema operativo principal, ya que cualquier error o cambio quedó aislado dentro de la VM. Además, este entorno facilitó la simulación de un escenario real de red entre cliente y servidor sin necesidad de utilizar hardware físico adicional.

Paso 2: Intento de escaneo de puertos (Reconocimiento):

Comando:

sudo nmps -sV -p- -T4 159.223.152.22 (Nota: El comando correcto es nmap)



```
admnyc@debian-ici:~$ sudo nmap -sV -p- -T4 159.223.152.22
[sudo] contraseña para admnyc:
sudo: nmap: command not found
admnyc@debian-ici:~$ sudo nmap -sV -p- -T4 159.223.152.22
sudo: nmap: command not found
```

Figura 2. Error de comando no encontrado al ejecutar Nmap.

Se intentó ejecutar un escaneo de puertos con nmap desde el servidor Debian hacia su propia IP pública, pero el sistema mostró el error command not found porque la herramienta no estaba instalada.

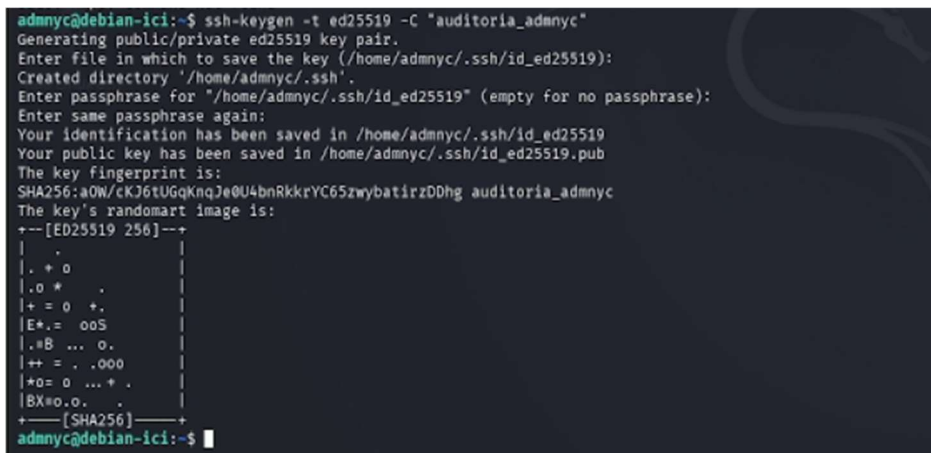
Explicación

El escaneo debía realizarse desde una máquina externa, como Kali Linux, y no desde el mismo servidor. Además, que nmap no esté instalado es una buena práctica de seguridad, ya que reduce herramientas que podrían ser usadas por un atacante en caso de comprometer el sistema.

Paso 3: Generación de par de llaves SSH (Autenticación Robusta):

Comando:

```
ssh-keygen -t ed25519 -C "auditoria_admnyc"
```



```
admnyc@debian-ici:~$ ssh-keygen -t ed25519 -C "auditoria_admnyc"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/admnyc/.ssh/id_ed25519):
Created directory '/home/admnyc/.ssh'.
Enter passphrase for '/home/admnyc/.ssh/id_ed25519' (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/admnyc/.ssh/id_ed25519
Your public key has been saved in /home/admnyc/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:a0W/cKJ6tUGqKnqJ0U4bnRkkrYC65zwybatirzDDhg auditoria_admnyc
The key's randomart image is:
+--[ED25519 256]--+
|  .                |
|..+ o              |
|.o *               |
|+= o +.            |
|E+.= ooS           |
|.B ... o.          |
|++ = . .ooo        |
|*o= o ... +.       |
|BX=o.o. .          |
+---[SHA256]-----+
admnyc@debian-ici:~$
```

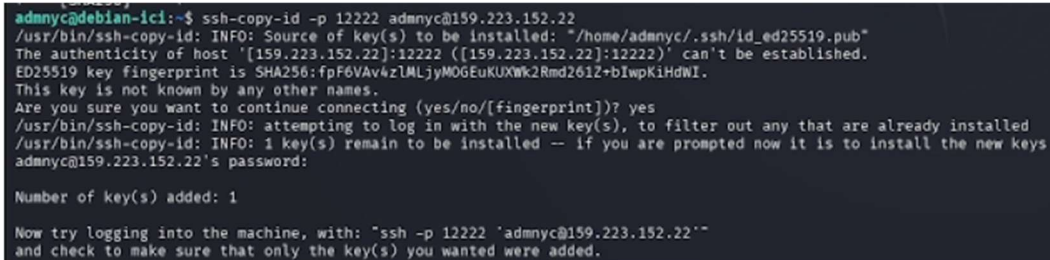
Figura 3. Creación de llaves SSH para autenticación robusta.

Se generó un par de llaves criptográficas para el usuario admnyc: una llave privada almacenada localmente y una llave pública destinada al servidor. Se utilizaron las opciones predeterminadas y no se configuró contraseña adicional para facilitar la automatización. Se utilizó el algoritmo ED25519 por ser uno de los métodos de cifrado más seguros y eficientes actualmente. Además, el uso de llaves criptográficas permite desactivar el acceso mediante contraseñas tradicionales, fortaleciendo la seguridad del servidor y reduciendo el riesgo de ataques por fuerza bruta.

Paso 4: Copia e instalación de la llave pública en el servidor

Comando:

```
ssh-copy-id -p 12222 admnyc@159.223.152.22
```



```
admnc@debian-ici:~$ ssh-copy-id -p 12222 admnyc@159.223.152.22
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/admnc/.ssh/id_ed25519.pub"
The authenticity of host '[159.223.152.22]:12222 ([159.223.152.22]:12222)' can't be established.
ED25519 key fingerprint is SHA256:fpF6VAv4zLMLjyMOGEuKUXWk2Rmd261Z+bIwpKiHdWI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
admnc@159.223.152.22's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -p 12222 'admnc@159.223.152.22'"
and check to make sure that only the key(s) you wanted were added.
```

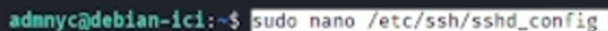
Figura 4. Almacenamiento de llave SSH en el host remoto.

Se instaló automáticamente la llave pública en el servidor remoto mediante ssh-copy-id usando el puerto 12222. El sistema confirmó la conexión y agregó correctamente la llave autorizada. Se utilizó ssh-copy-id porque permite copiar la llave pública de forma segura al archivo `authorized_keys` del servidor. Con esto, el acceso puede realizarse mediante autenticación por llaves criptográficas, evitando el uso de contraseñas y mejorando la seguridad del sistema.

Paso 5: Modificación de la configuración del servidor SSH:

Comando:

```
sudo nano /etc/ssh/sshd_config
```



```
admnc@debian-ici:~$ sudo nano /etc/ssh/sshd_config
```

Figura 5. Acceso a los parámetros de configuración de SSH.

Se abrió el archivo principal de configuración del servicio SSH (`sshd_config`) utilizando el editor Nano con permisos de administrador. Este archivo permite modificar parámetros de seguridad del acceso remoto, como cambiar el puerto por defecto y deshabilitar el acceso directo del usuario root. Se utilizó sudo porque el archivo se encuentra en una ruta protegida del sistema y requiere privilegios elevados para realizar cambios.

Paso 6: Mitigación de ataques de fuerza bruta por cambio de puerto por defecto:

Archivo modificado:

/etc/ssh/sshd_config (Línea: Port 12222)



```
Include /etc/ssh/sshd_config.d/*.conf

Port 12222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
```

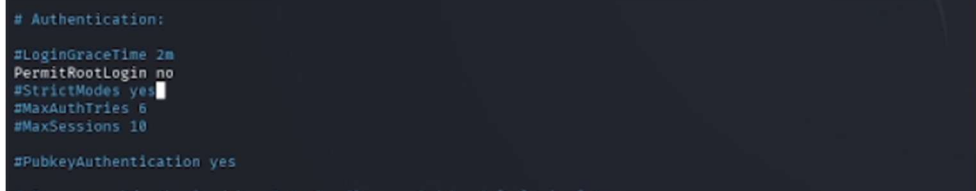
Figura 6. Configuración de puerto SSH alternativo para mayor seguridad.

Se modificó el archivo de configuración de SSH para cambiar el puerto predeterminado 22 por el puerto personalizado 12222, y posteriormente se guardaron los cambios. El cambio de puerto ayuda a reducir intentos automáticos de ataque dirigidos al puerto 22, que es el más utilizado por defecto en SSH. Con esto, se mejora la seguridad del servidor al dificultar accesos no autorizados y obligar a especificar el nuevo puerto para conectarse.

Paso 7: Prohibir explícitamente el login del superusuario (Mitigación de escalación de privilegios):

Archivo modificado:

/etc/ssh/sshd_config (Línea: PermitRootLogin no)



```
# Authentication:
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes
```

Figura 7. Restricción de acceso SSH al usuario administrador root.

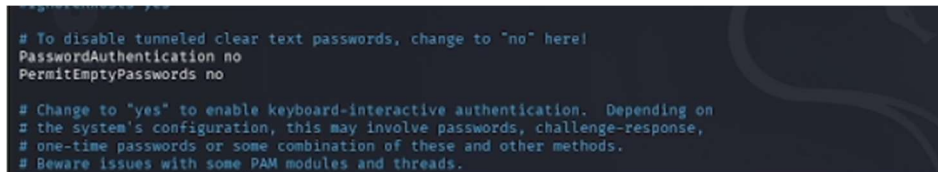
Se modificó la directiva PermitRootLogin en la configuración de SSH, activándola y estableciendo su valor en no para deshabilitar el acceso remoto directo del usuario root. Esta configuración evita que el superusuario pueda iniciar sesión directamente desde SSH, reduciendo riesgos de ataques contra la cuenta root. Ahora, la

administración del servidor debe realizarse mediante un usuario normal con permisos sudo, permitiendo mayor control y registro de actividades.

Paso 8: Restricción absoluta de autenticación por contraseña tradicional:

Archivo modificado:

/etc/ssh/sshd_config (Líneas: PasswordAuthentication no y PermitEmptyPasswords no)



```
# To disable tunneled clear text passwords, change to "no" here!  
PasswordAuthentication no  
PermitEmptyPasswords no  
  
# Change to "yes" to enable keyboard-interactive authentication. Depending on  
# the system's configuration, this may involve passwords, challenge-response,  
# one-time passwords or some combination of these and other methods.  
# Beware issues with some PAM modules and threads.
```

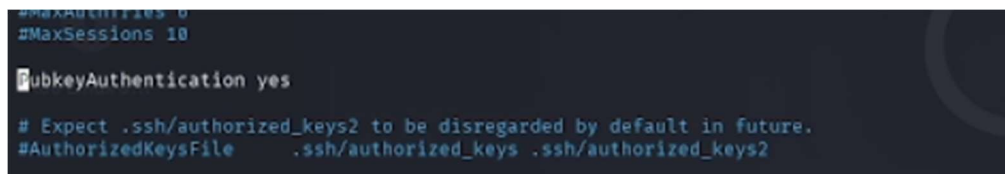
Figura 8. Restricción de acceso SSH mediante contraseñas.

Se modificaron las directivas de autenticación en la configuración de SSH para deshabilitar el acceso mediante contraseñas y bloquear cuentas sin contraseña. Con estas configuraciones, el servidor solo permite autenticación mediante llaves criptográficas, aumentando la seguridad del acceso remoto. Además, se evita el uso de contraseñas vacías y se bloquean ataques de fuerza bruta basados en el intento masivo de contraseñas.

Paso 9: Forzar autenticación criptográfica por llaves públicas:

Archivo modificado:

/etc/ssh/sshd_config (Línea: PubkeyAuthentication yes)



```
#MaxAuthTries 6  
#MaxSessions 10  
  
PubkeyAuthentication yes  
  
# Expect .ssh/authorized_keys2 to be disregarded by default in future.  
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
```

Figura 9. Habilitación explícita de la autenticación por llave pública.

Se activó la directiva PubkeyAuthentication en la configuración de SSH y se estableció su valor en yes para permitir la autenticación mediante llaves criptográficas. Esta

configuración habilita el acceso seguro usando las llaves previamente generadas. De esta manera, el servidor verifica automáticamente la identidad del usuario mediante el archivo `authorized_keys`, evitando el uso de contraseñas y reforzando la seguridad del acceso remoto.

Paso 10: Control de persistencia y desconexión por inactividad (Defensa en profundidad):

Archivo modificado:

`/etc/ssh/sshd_config` (Líneas: `ClientAliveInterval 300` y `ClientAliveCountMax 2`)

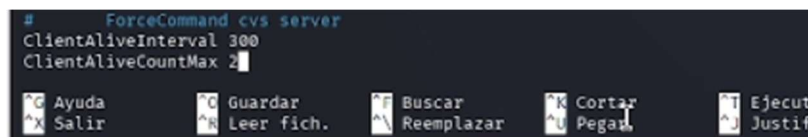


Figura 10. Configuración de tiempos de desconexión por inactividad.

Se configuraron parámetros de tiempo de espera en SSH para cerrar automáticamente sesiones inactivas después de cierto periodo. Estas reglas permiten que el servidor detecte sesiones sin actividad y las cierre automáticamente tras varios minutos de inactividad. Con ello, se reduce el riesgo de accesos no autorizados en equipos que hayan quedado abiertos o desatendidos.

Se guardaron los cambios realizados en el archivo de configuración utilizando `Ctrl + O`, se confirmó con `Enter` y posteriormente se cerró el editor Nano con `Ctrl + X`.

Paso 11: Aplicación de cambios en SSH y endurecimiento perimetral mediante Firewall (UFW):

Comandos ejecutados:

```
sudo sshd -t && sudo systemctl restart ssh  
  
sudo ufw default deny incoming  
  
sudo ufw default allow outgoing  
  
sudo ufw allow 12222/tcp comment 'Acceso SSH Administrativo'
```

sudo ufw enable

```
admny@debian-ici:~$ sudo nano /etc/ssh/sshd_config
admny@debian-ici:~$ sudo sshd -t && sudo systemctl restart ssh
admny@debian-ici:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
admny@debian-ici:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
admny@debian-ici:~$ sudo ufw allow 12222/tcp comment 'Acceso SSH Administrativo'
Rule added
Rule added (v6)
admny@debian-ici:~$ sudo ufw enable
Firewall is active and enabled on system startup
```

Figura 11. Reinicio de SSH y endurecimiento perimetral con UFW.

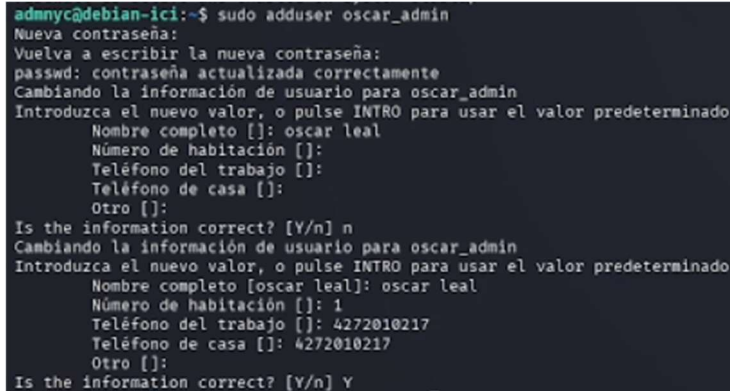
Se verificó la configuración del servicio SSH para comprobar que no existieran errores antes de reiniciarlo. Después, se configuró el firewall ufw estableciendo reglas de seguridad restrictivas: bloquear todas las conexiones entrantes por defecto, permitir conexiones salientes y habilitar únicamente el acceso al puerto 12222. Finalmente, se activó el firewall para que las reglas quedaran funcionando de manera permanente.

- Primero se validó el archivo de configuración de SSH para evitar errores que pudieran impedir futuras conexiones al servidor. Una vez comprobado, el servicio fue reiniciado para aplicar los cambios realizados.
- Posteriormente, se configuró el firewall siguiendo una política de “todo bloqueado excepto lo necesario”. Esto permite reducir la superficie de ataque, ya que cualquier conexión no autorizada será rechazada automáticamente antes de llegar al sistema.
- También se permitió el tráfico saliente para que el servidor pueda seguir descargando actualizaciones y paquetes de seguridad. Además, únicamente se abrió el puerto 12222, utilizado para la administración remota mediante SSH.
- Finalmente, al habilitar ufw, el firewall quedó activo y configurado para iniciarse automáticamente cada vez que el servidor se reinicie, garantizando la permanencia de las medidas de seguridad aplicadas.

Paso 12: Creación de usuarios nominales independientes para administración:

Comando:

sudo adduser oscar_admin

A screenshot of a terminal window showing the execution of the 'adduser' command. The prompt is 'admny@debian-ici:~\$'. The command 'sudo adduser oscar_admin' is entered. The process continues with prompts for a new password, confirmation of the password, and then a series of questions to collect user information: 'Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado'. The user provides 'oscar leal' for the full name, '1' for the room number, '4272010217' for the work phone, and '4272010217' for the home phone. The system asks 'Is the information correct? [Y/n]' and the user responds 'n'. The process repeats for the work phone and home phone, where the user provides '4272010217' and the system asks 'Is the information correct? [Y/n]' and the user responds 'Y'.

```
admny@debian-ici:~$ sudo adduser oscar_admin
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para oscar_admin
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo []: oscar leal
Número de habitación []:
Teléfono del trabajo []:
Teléfono de casa []:
Otro []:
Is the information correct? [Y/n] n
Cambiando la información de usuario para oscar_admin
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo [oscar leal]: oscar leal
Número de habitación []: 1
Teléfono del trabajo []: 4272010217
Teléfono de casa []: 4272010217
Otro []:
Is the information correct? [Y/n] Y
```

Figura 12. Creación de un usuario administrador nominal en el sistema.

Se creó una nueva cuenta de usuario llamada oscar_admin utilizando el comando adduser con privilegios de administrador. Durante el proceso se asignó una contraseña y se registraron los datos básicos del usuario.

La creación de cuentas individuales permite mantener un mejor control y registro de las actividades realizadas en el servidor.

De esta forma, cada acción queda asociada a un usuario específico dentro de los archivos de eventos del sistema. Además, separar las cuentas administrativas mejora la seguridad y la administración de accesos, ya que permite habilitar o deshabilitar usuarios de manera individual sin afectar al resto del equipo.

Paso 13: Intento de asignación de privilegios y auditoría de comandos mediante ayuda nativa:

Comando ejecutado:

sudo usermod -a6 sudo oscar_admin

```
admny@debian-ici:~$ sudo usermod -a6 sudo oscar_admin
usermod: opción inválida -- '6'
Modo de uso: usermod [opciones] USUARIO

Opciones:
-a, --append                append the user to the supplemental GROUPS
                             mentioned by the -G option without removing
                             the user from other groups
-b, --badname               allow bad names (DEPRECATED)
-c, --comment COMENTARIO    nuevo valor del campo GECOS
-d, --home DIR_PERSONAL     nuevo directorio personal del nuevo usuario
-e, --expiredate FECHA_EXPIR establece la fecha de caducidad de la
                             cuenta a FECHA_EXPIR
-f, --inactive INACTIVO     establece el tiempo de inactividad después
                             de que caduque la cuenta a INACTIVO
-g, --gid GRUPO             fuerza el uso de GRUPO para la nueva cuenta
                             de usuario
-G, --groups GRUPOS         lista de grupos suplementarios
-h, --help                  muestra este mensaje de ayuda y termina
-l, --login NOMBRE          nuevo nombre para el usuario
-L, --lock                  bloquea la cuenta de usuario
-m, --move-home             mueve los contenidos del directorio
                             personal al directorio nuevo (usar sólo
                             junto con -d)
-o, --non-unique            permite usar UID duplicados (no únicos)
-p, --password CONTRASEÑA   usar la contraseña cifrada para la nueva cuenta
-P, --prefix PREFIX_DIR     prefix directory where are located the /etc/* files
-r, --remove                remove the user from only the supplemental GROUPS
                             mentioned by the -G option without removing
                             the user from other groups
-R, --root DIR_CHROOT       establece DIR_CHROOT como el directorio
                             al cual hacer chroot
-s, --shell CONSOLA         nueva consola de acceso para la cuenta del
                             usuario
-u, --uid UID               fuerza el uso del UID para la nueva cuenta
                             de usuario
-U, --unlock                desbloquea la cuenta de usuario
-v, --add-subuids FIRST-LAST add range of subordinate uids
-V, --del-subuids FIRST-LAST remove range of subordinate uids
-w, --add-subgids FIRST-LAST add range of subordinate gids
-W, --del-subgids FIRST-LAST remove range of subordinate gids
-Z, --selinux-user SEUSER   new SELinux user mapping for the user account
                             -selinux-range SERANGE new SELinux MLS range for the user account

admny@debian-ici:~$
```

Figura 13. Error de sintaxis y despliegue de ayuda en usermod.

Se intentó agregar al usuario oscar_admin al grupo sudo para otorgarle permisos administrativos. Durante el proceso, el sistema mostró un error de sintaxis debido a que se ingresó un parámetro incorrecto en el comando. El error ocurrió porque se escribió un valor no válido en lugar de las opciones correctas del comando usermod. Gracias al mensaje mostrado por el sistema, fue posible identificar las banderas adecuadas para añadir usuarios a grupos sin afectar su configuración actual.

Este tipo de errores son comunes en la administración de servidores y es importante interpretar correctamente los mensajes de la terminal para corregir problemas sin comprometer la estabilidad o seguridad del sistema.

Paso 14: Asignación de privilegios elevados y verificación de grupos de seguridad:

Comandos ejecutados:

```
sudo usermod -aG sudo oscar_admin
```

groups oscar_admin



```
admny@debian-1ci:~$ sudo usermod -aG sudo oscar_admin
admny@debian-1ci:~$ groups oscar_admin
oscar_admin : oscar_admin sudo users
admny@debian-1ci:~$
```

Figura 14. Inclusión exitosa del usuario en el grupo sudo.

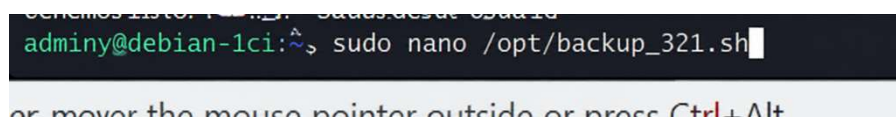
Se corrigió el comando usando la opción -aG para agregar al usuario oscar_admin al grupo sudo. Después, se verificó el cambio con el comando groups.

Al agregar el usuario al grupo sudo, se le otorgaron permisos administrativos de forma segura usando sus propias credenciales. Finalmente, la verificación confirmó que oscar_admin ya cuenta con privilegios de administrador dentro del sistema.

Paso 15: Creación y automatización del script de respaldos (Hardering MariaDB /PostgreSQL y el script 3-2-1):

Comando:

sudo nano /opt/backup_321.sh



```
admny@debian-1ci:~$ sudo nano /opt/backup_321.sh
```

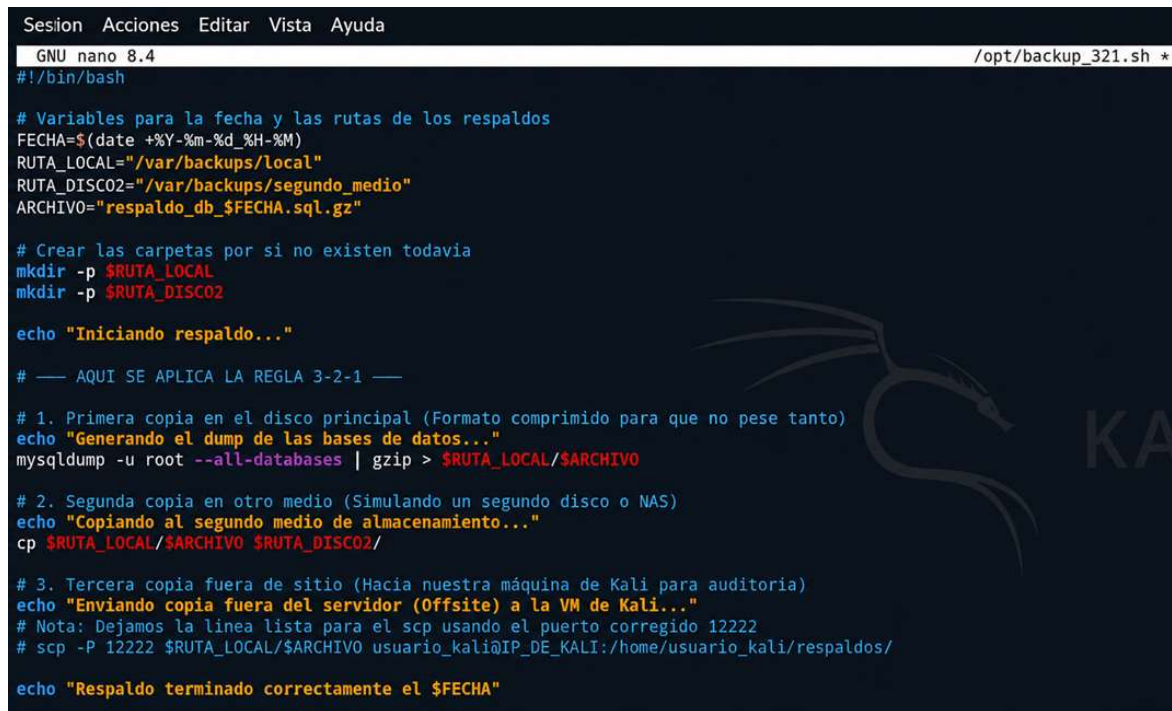
Figura 15. Apertura del script de respaldos con el editor nano.

Se creó un archivo de script Bash llamado backup_321.sh dentro del directorio /opt/ utilizando el editor Nano con privilegios de administrador. El directorio /opt/ se utiliza para almacenar scripts y aplicaciones adicionales, manteniendo una mejor organización del sistema. Además, el nombre del script hace referencia a la estrategia de respaldo 3-2-1 utilizada en seguridad informática. El archivo fue creado con sudo para que solo usuarios con permisos administrativos puedan modificarlo, protegiendo así el script de accesos o cambios no autorizados.

Paso 16: Desarrollo del Script de RespalDOS aplicando la Regla Estricta 3-2-1:

Archivo editado:

/opt/backup_321.sh



```
Sesión Acciones Editar Vista Ayuda
GNU nano 8.4 /opt/backup_321.sh *
#!/bin/bash

# Variables para la fecha y las rutas de los respaldos
FECHA=$(date +%Y-%m-%d_%H-%M)
RUTA_LOCAL="/var/backups/local"
RUTA_DISC02="/var/backups/segundo_medio"
ARCHIVO="respaldo_db_$FECHA.sql.gz"

# Crear las carpetas por si no existen todavia
mkdir -p $RUTA_LOCAL
mkdir -p $RUTA_DISC02

echo "Iniciando respaldo..."

# --- AQUI SE APLICA LA REGLA 3-2-1 ---

# 1. Primera copia en el disco principal (Formato comprimido para que no pese tanto)
echo "Generando el dump de las bases de datos..."
mysqldump -u root --all-databases | gzip > $RUTA_LOCAL/$ARCHIVO

# 2. Segunda copia en otro medio (Simulando un segundo disco o NAS)
echo "Copiando al segundo medio de almacenamiento..."
cp $RUTA_LOCAL/$ARCHIVO $RUTA_DISC02/

# 3. Tercera copia fuera de sitio (Hacia nuestra máquina de Kali para auditoria)
echo "Enviando copia fuera del servidor (Offsite) a la VM de Kali..."
# Nota: Dejamos la linea lista para el scp usando el puerto corregido 12222
# scp -P 12222 $RUTA_LOCAL/$ARCHIVO usuario_kali@IP_DE_KALI:/home/usuario_kali/respaldos/

echo "Respaldo terminado correctamente el $FECHA"
```

Figura 16. Programación del script de respaldos bajo regla 3-2-1.

Se utilizaron variables dinámicas para que cada respaldo generado tenga una fecha y hora únicas, evitando que los archivos anteriores sean reemplazados y permitiendo recuperar información de momentos específicos.

La estrategia 3-2-1 se implementó mediante tres copias del respaldo:

- **Primera copia:** se generó y comprimió la base de datos en el disco principal utilizando mysqldump y gzip, reduciendo el tamaño del archivo y optimizando espacio.
- **Segunda copia:** el archivo comprimido se duplicó en otra ubicación de almacenamiento, simulando un segundo disco o servidor de respaldo.
- **Tercera copia:** se dejó preparada una transferencia segura mediante scp hacia una máquina externa, permitiendo contar con un respaldo fuera del servidor principal en caso de fallos graves o desastres.

Se guardaron los cambios realizados en el archivo de configuración utilizando Ctrl + O, se confirmó con Enter y posteriormente se cerró el editor Nano con Ctrl + X.

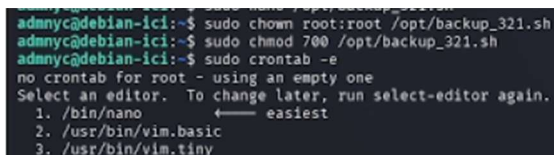
Paso 17: Endurecimiento de permisos (*File Permissions Hardening*) y programación de tareas de respaldo:

Comandos ejecutados:

```
sudo chown root:root /opt/backup_321.sh
```

```
sudo chmod 700 /opt/backup_321.sh
```

```
sudo crontab -e
```



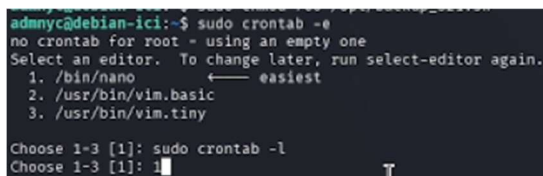
```
admny@debian-ici:~$ sudo chown root:root /opt/backup_321.sh
admny@debian-ici:~$ sudo chmod 700 /opt/backup_321.sh
admny@debian-ici:~$ sudo crontab -e
no crontab for root - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          ← easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
```

Figura 17. Restricción de permisos y apertura del programador crontab.

Paso 18: Selección del editor predeterminado para tareas programadas y auditoría de errores de entrada:

Comando / Acción:

Selección de opción 1 (/bin/nano) en el selector de entorno de Cron.



```
admny@debian-ici:~$ sudo crontab -e
no crontab for root - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano          ← easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny

Choose 1-3 [1]: sudo crontab -l
Choose 1-3 [1]: 1
```

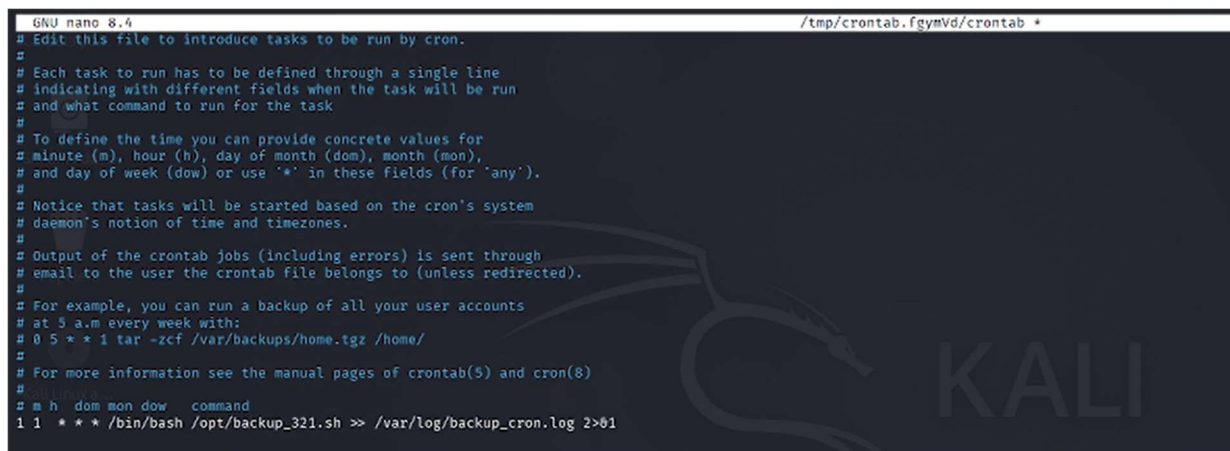
Figura 18. Selección del editor nano y error de entrada en terminal.

Al ejecutar crontab por primera vez como usuario root, el sistema solicitó seleccionar un editor de texto predeterminado. Después de un intento incorrecto en la terminal, se eligió la opción 1 para configurar el editor Nano. Esto se realizó para definir un editor sencillo y fácil de utilizar al administrar tareas programadas con Cron. Además, el sistema mostró estabilidad al ignorar el comando incorrecto sin afectar el servicio, permitiendo continuar con la configuración normalmente.

Paso 19: Configuración de la tarea programada y redireccionamiento de registros (Logs):

Archivo modificado: crontab (Perfil de usuario root)

```
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
```



```
GNU nano 8.4 /tmp/crontab.fgymVd/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
```

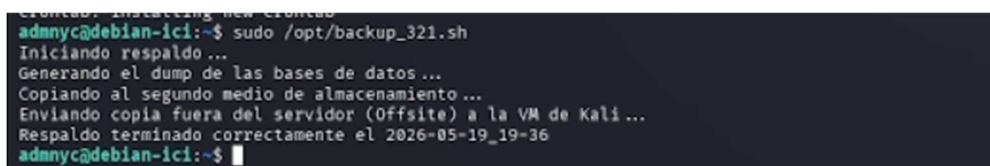
Figura 19. Programación de la tarea cron y redirección de logs.

Se agregó una tarea programada en Cron para ejecutar automáticamente el script de respaldo mediante `/bin/bash` todos los días a la 1:01 AM. Además, se configuró el almacenamiento de mensajes y errores en un archivo de registro. La regla `1 1 * * *` indica que el respaldo se ejecutará diariamente a la 1:01 de la madrugada, horario en el que el servidor suele tener menor carga de trabajo. También se utilizaron rutas absolutas para asegurar que Cron encuentre correctamente tanto el intérprete de Bash como el script de respaldo. Finalmente, se redirigieron las salidas y errores al archivo `/var/log/backup_cron.log`, permitiendo mantener un historial y facilitar la detección de fallos o problemas durante la ejecución automática de los respaldos.

Paso 20: Fase de pruebas, validación de ejecución del script y estampa de tiempo:

Comando:

```
sudo /opt/backup_321.sh
```



```
admnyc@debian-ici:~$ sudo /opt/backup_321.sh
Iniciando respaldo ...
Generando el dump de las bases de datos ...
Copiando al segundo medio de almacenamiento ...
Enviando copia fuera del servidor (Offsite) a la VM de Kali ...
Respaldo terminado correctamente el 2026-05-19_19-36
admnyc@debian-ici:~$
```

Figura 20. Ejecución de prueba y validación del script de respaldo.

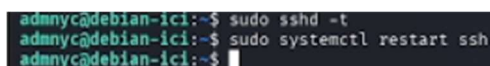
Se ejecutó manualmente el script de respaldos con sudo para comprobar que funcionara correctamente antes de automatizarlo con Cron. La prueba permitió verificar permisos, rutas y el funcionamiento del script. Además, la salida confirmó la creación de las tres copias de respaldo definidas en la estrategia 3-2-1 y validó que los archivos se guardan con fecha y hora para mantener un orden cronológico.

Paso 21: Segunda validación de integridad y reaplicación de políticas en el demonio SSH:

Comandos ejecutados:

sudo sshd -t

sudo systemctl restart ssh



```
admny@debian-ici:~$ sudo sshd -t
admny@debian-ici:~$ sudo systemctl restart ssh
admny@debian-ici:~$
```

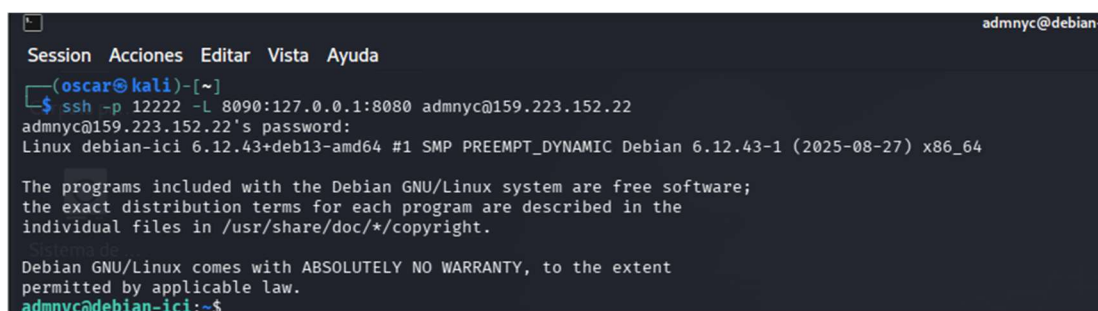
Figura 21. Auditoría de sintaxis y reinicio del daemon sshd.

Se volvió a validar la configuración del servicio SSH y posteriormente se reinició el servicio utilizando systemctl para aplicar los cambios realizados. La validación permitió comprobar que la configuración no tuviera errores antes del reinicio. Después, al reiniciar el servicio SSH, se aseguraron los nuevos cambios y configuraciones aplicadas, permitiendo que las futuras conexiones funcionen correctamente y de forma segura.

Comandos de revisión (Todo adentro del servidor):

Desde tu Kali Linux (Abrir el túnel y conectarte):

ssh -p 12222 -L 8090:127.0.0.1:8080 admnyc@159.223.152.22



```
admny@kali:~$ ssh -p 12222 -L 8090:127.0.0.1:8080 admnyc@159.223.152.22
admny@159.223.152.22's password:
Linux debian-ici 6.12.43+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.43-1 (2025-08-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
admny@debian-ici:~$
```

Figura 22. Apertura de túnel local y conexión remota SSH.

Justificación del uso de Túnel SSH

En esta práctica se implementó un túnel SSH como medida de seguridad para proteger los servicios internos del servidor. En lugar de dejar puertos como el 8080 expuestos a internet, se mantuvieron bloqueados mediante el firewall, permitiendo el acceso únicamente a través de una conexión SSH autenticada.

El firewall (UFW) mantiene una política restrictiva bloqueando el acceso externo a dichos puertos, reduciendo así la superficie de ataque del servidor. Además, todo el flujo de datos entre el cliente (Kali Linux) y el servidor viaja cifrado mediante el protocolo SSH, evitando ataques de interceptación de datos tipo Man-in-the-Middle.

Finalmente, este método garantiza que solo los administradores autorizados que hayan superado la autenticación por llaves criptográficas puedan interactuar con los servicios internos del servidor.

1. Revisar el Firewall (UFW):

sudo ufw status verbose

```
admnyc@debian-ici:~$ sudo ufw status verbose
[sudo] contraseña para admnyc:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
12222 ALLOW IN Anywhere
8080 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
12222/tcp ALLOW IN Anywhere # Acceso SSH Administrativo
12222 (v6) ALLOW IN Anywhere (v6)
8080 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)
12222/tcp (v6) ALLOW IN Anywhere (v6) # Acceso SSH Administrativo

admnyc@debian-ici:~$
```

Figura 23. Auditoría del estado y reglas activas en UFW.

El comando `sudo ufw status verbose` muestra de forma detallada el estado del firewall, incluyendo políticas globales y reglas activas. Permite verificar qué puertos y protocolos están abiertos, como 12222/tcp para acceso SSH administrativo.

2. Revisar la Cuenta de Usuario Nueva:

groups oscar_admin

```
admny@debian-ici:~$ groups oscar_admin
oscar_admin : oscar_admin sudo users
```

Figura 24. Verificación de grupos asociados al nuevo usuario.

El comando `groups oscar_admin` permite verificar los grupos y privilegios asignados a la cuenta en el sistema. La salida confirma que el usuario pertenece al grupo `sudo`, habilitando la ejecución de tareas administrativas de forma controlada.

3. Revisar el Respaldo Automático (Cron):

sudo crontab -l

```
admny@debian-ici:~$ sudo crontab -l
[sudo] contraseña para admny:
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
admny@debian-ici:~$
```

Figura 25. Listado y verificación de tareas cron activas.

El comando `sudo crontab -l` permite visualizar las tareas programadas del usuario `root` en el servicio Cron. Se utiliza para verificar que el script de respaldos y sus registros hayan sido configurados correctamente para su ejecución automática.

4. Revisar que los Archivos de Respaldo ya existen:

ls -lh /var/backups/local/

ls -lh /var/backups/segundo_medio/

```
admnyc@debian-ici:~$ ls -lh /var/backups/local/
ls -lh /var/backups/segundo_medio/
total 1.1M
-rw-r--r-- 1 root root 518K may 19 19:36 respaldo_db_2026-05-19_19-36.sql.gz
-rw-r--r-- 1 root root 518K may 20 01:01 respaldo_db_2026-05-20_01-01.sql.gz
total 1.1M
-rw-r--r-- 1 root root 518K may 19 19:36 respaldo_db_2026-05-19_19-36.sql.gz
-rw-r--r-- 1 root root 518K may 20 01:01 respaldo_db_2026-05-20_01-01.sql.gz
admnyc@debian-ici:~$
```

Figura 26. Inspección de directorios de almacenamiento y copias generadas.

El comando `ls -lh` permite listar archivos con detalles y tamaños legibles en los directorios de respaldos. Se utiliza para comprobar que los archivos `.sql.gz` fueron generados correctamente y que las copias mantienen el mismo tamaño y fecha de creación.

5. Revisar el Hardening de SSH (El blindaje):

`sudo grep -E "PermitRootLogin|PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config`

```
admnyc@debian-ici:~$ sudo grep -E "PermitRootLogin|PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
# PasswordAuthentication. Depending on your PAM configuration,
# the setting of "PermitRootLogin yes
# PAM authentication, then enable this but set PasswordAuthentication
admnyc@debian-ici:~$
```

Figura 27. Auditoría de directivas de endurecimiento en sshd_config.

El comando `sudo grep -E`

`"PermitRootLogin|PasswordAuthentication|PubkeyAuthentication"`

`/etc/ssh/sshd_config` permite verificar las configuraciones críticas de seguridad de SSH, confirmando que el acceso root y la autenticación por contraseña están deshabilitados y que únicamente se permite el acceso mediante llaves criptográficas.

6. Revisar el estado de las Bases de Datos (SQL):

Ver el estado de MariaDB - `sudo systemctl status mariadb`

Ver qué puertos están escuchando de forma interna y segura - `sudo ss -tulpn | grep -E "3306|5432|8080"`

```
admmc@debian-ici:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.0.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-06 22:59:15 EDT; 1 week 6 days ago
     Invocation: ad986b6a86d245da97e73cdcbb14852
       Docs: man:mariadb(8)
             https://mariadb.com/kb/en/library/systemd/
   Main PID: 27647 (mariadbd)
    Status: "Taking your SQL requests now..."
     Tasks: 9 (limit: 7226)
    Memory: 141.8M (peak: 142.3M)
       CPU: 2min 49.315s
    CGroup: /system.slice/mariadb.service
            └─27647 /usr/sbin/mariadbd

may 06 22:59:15 debian-ici mariadbd[27647]: 2026-05-06 22:59:15 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
may 06 22:59:15 debian-ici mariadbd[27647]: 2026-05-06 22:59:15 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
may 06 22:59:15 debian-ici mariadbd[27647]: 2026-05-06 22:59:15 0 [Note] /usr/sbin/mariadbd: ready for connections.
may 06 22:59:15 debian-ici mariadbd[27647]: Version: '11.0.6-MariaDB-0-deb13ui from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help get to 10k stars at https://github.com/MariaDB/Server
may 06 22:59:15 debian-ici systemd[1]: Started mariadb.service - MariaDB 11.0.6 database server.
may 06 22:59:15 debian-ici /etc/mysql/debian-start[27664]: Upgrading MariaDB tables if necessary.
may 06 22:59:15 debian-ici /etc/mysql/debian-start[27677]: Checking for insecure root accounts.
may 06 22:59:15 debian-ici /etc/mysql/debian-start[27681]: Triggering myisam-recover for all MyISAM tables and aria-recover for all Aria tables
may 08 20:08:13 debian-ici mariadbd[27647]: 2026-05-08 20:08:13 77 [Warning] Access denied for user 'apiadmin'@'localhost' (using password: YES)
may 19 20:08:39 debian-ici mariadbd[27647]: 2026-05-19 20:08:39 98 [Warning] Access denied for user 'root'@'localhost'

admmc@debian-ici:~$ sudo ss -tulpn | grep -E "3306|5432|8080"
tcp        LISTEN 0      80      127.0.0.1:3306      0.0.0.0:*      users:((("mariadbd",pid=27647,fd=28))
tcp        LISTEN 0      511     0.0.0.0:8080       0.0.0.0:*      users:((("nginx",pid=77869,fd=5),("nginx",pid=77866,fd=5)))
admmc@debian-ici:~$
```

Figura 28. Auditoría del servicio MariaDB y sockets de red activos.

Los comandos `sudo systemctl status mariadb` y `sudo ss -tulpn | grep -E "3306|5432|8080"` permiten verificar que el servicio MariaDB se encuentra activo y que el puerto 3306 está escuchando únicamente en 127.0.0.1, garantizando su funcionamiento interno sin exposición directa a redes externas.

7. Probar el túnel (Desde tu Kali Linux otra vez):

Para cerrar la prueba demostrando que el túnel que abriste en el Paso 1 funciona:

- Abre otra pestaña o ventana de la terminal en tu Kali Linux (esta no debe estar conectada al servidor, debe decir kali@kali).
- Haz un escaneo rápido al puerto local 8090 de tu Kali para demostrar que estás mapeando el puerto 8080 del servidor de forma segura:

```
ooscar@kali: ~
Session Acciones Editar Vista Ayuda
(ooscar@kali)-[~]
└─$ curl -I http://127.0.0.1:8090
HTTP/1.1 400 Bad Request
Server: nginx
Date: Wed, 20 May 2026 17:59:12 GMT
Content-Type: text/html
Content-Length: 248
Connection: close

(ooscar@kali)-[~]
└─$
```

Figura 29. Prueba local de conectividad HTTP mediante comando curl.

Para comprobar el funcionamiento del firewall y del túnel SSH, se realizó una petición HTTP con curl -I desde Kali Linux hacia 127.0.0.1:8090. La respuesta recibida del servidor nginx confirmó que el tráfico viajó correctamente a través del túnel SSH cifrado.

Aunque se obtuvo un código HTTP 400 (Bad Request), esto indica que la conexión con el servicio interno fue exitosa y que el puerto remoto 8080 es accesible únicamente mediante el túnel seguro, permaneciendo bloqueado para el acceso público gracias a las reglas del firewall.

Con esta prueba se valida que la arquitectura implementada reduce la superficie de ataque y permite administrar el servicio de forma segura sin exponer puertos innecesarios a internet.

¿Un usuario puede entrar con una contraseña y si no por qué?

No, el usuario no puede entrar con contraseña porque el servicio SSH fue configurado para permitir únicamente autenticación mediante llaves criptográficas públicas y privadas. Esto se logró deshabilitando la opción PasswordAuthentication no en el archivo de configuración de SSH.

Gracias a esta medida de hardening, el servidor rechaza cualquier intento de acceso con contraseña, aumentando la seguridad al evitar ataques de fuerza bruta, robo de credenciales y uso de contraseñas débiles.